

Data Structure ver_2.0

1. 데이터 구조 개요

이 문서는 Just Pick It 시스템에서 사용하는 주요 DB 테이블, ENUM, 테이블 간 관계를 정의한다.

Control Server는 주문, 재고, 로봇 상태, 작업 상태, 예외 기록을 저장하고 UI에 제공한다.

Fleet Manager는 Control Server에 저장된 주문/상품/로봇 정보를 기반으로 task를 생성하고, task 실행 결과를 Control Server에 보고한다.

발표/시나리오 관점에서는 PICKY를 “AMR + Cobot이 함께 동작하는 작업 세트”로 설명한다.

실제 구현/DB 관점에서는 PICKY와 COBOT을 개별 robot으로 저장하고, robot_unit으로 두 로봇의 작업 세트를 연결한다.

주문에 포함된 상품 목록은 order_item에 저장한다.

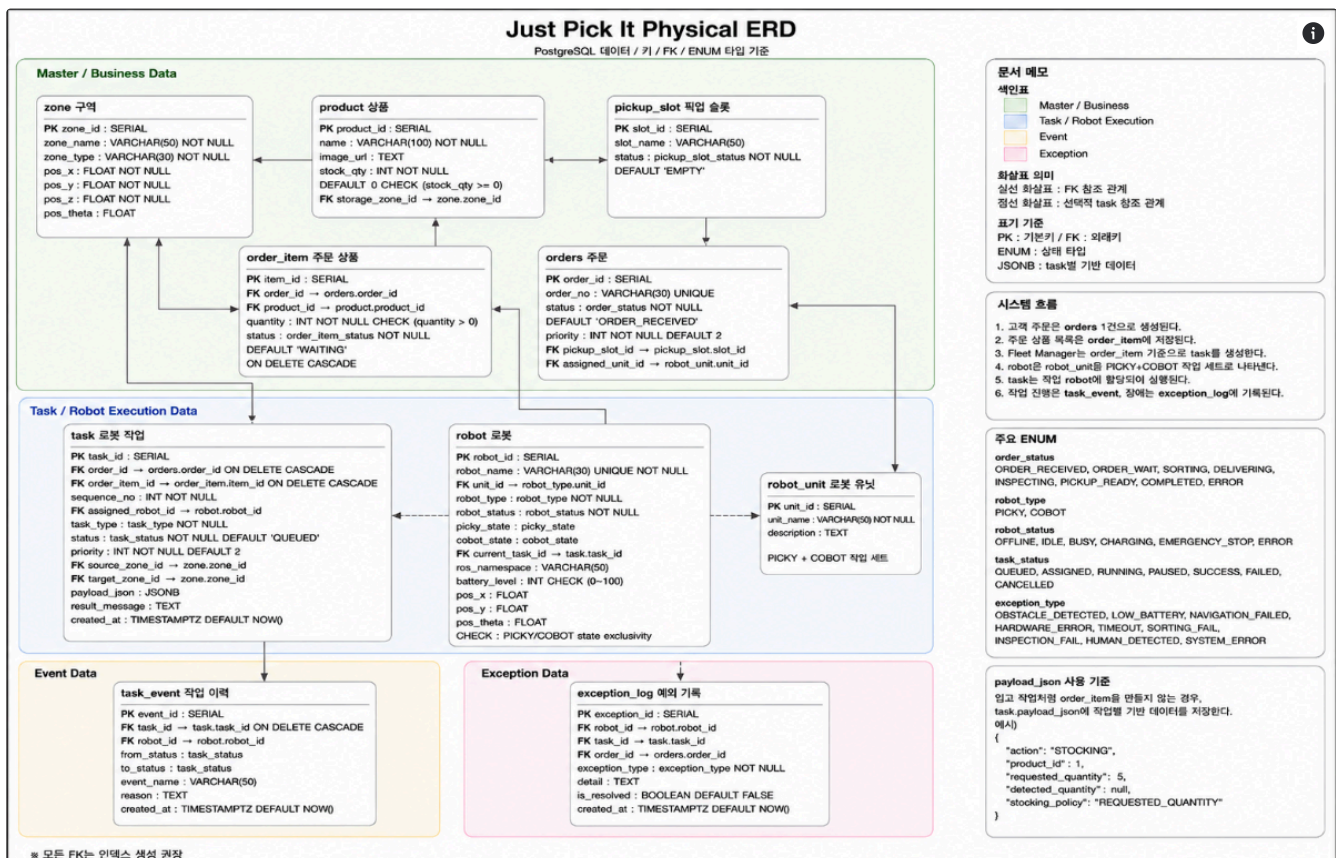
Fleet Manager가 생성한 task는 order_item_id를 통해 특정 상품과 연결된다.

따라서 order_item은 “주문 원본 상품 목록”이고, task는 “로봇이 실행할 작업 순서”이다.

입고는 주문과 다르게 order_item을 만들지 않는다.

입고 대상 상품, 요청 수량, 실제 감지 수량은 task.payload_json에 저장한다.

2. ERD 개요



핵심은 고객 주문을 Fleet Manager가 실행 가능한 robot task로 분해하고, 각 task를 PICKY와 COBOT의 실제 상태 변화와 연결해 관리하는 구조이다.

Control Server는 주문, 재고, 로봇 상태, task 상태, 예외 기록을 저장하고 UI에 보여준다.

Fleet Manager는 Task Manager와 Traffic Manager를 포함하며, 실제 task 생성, task 배정, 경로 충돌 판단, State Manager 명령 전달을 담당한다.

PICKY와 COBOT은 Control Server와 직접 통신하지 않고 Fleet Manager를 통해 제어된다.

3. 테이블 역할 요약

테이블	역할	사용 위치
zone	로봇이 이동하거나 작업하는 위치 정보	Fleet Manager, Traffic Manager, PICKY 목적지, COBOT 작업 위치
product	상품 정보와 재고	고객 상품 목록, 관리자 재고 관리
pickup_slot	고객 픽업 슬롯 상태	픽업존, 고객 수령
orders	주문 1건의 대표 상태	고객 주문 상태, 관리자 주문 현황
order_item	주문에 포함된 상품 목록	상품별 상자 task, 검수 기준
robot_unit	함께 동작하는 PICKY와 COBOT 묶음	Fleet Manager task 배정, 같은 세트 로봇 조회
robot	PICKY / COBOT 개별 로봇 상태	관리자 관제, Fleet Manager 상태 관리
task	Fleet Manager가 로봇에게 줄 작업 단위	작업 큐, task 배정, task 진행 상태
task_event	작업 진행 이력	관리자 작업 타임라인, 디버깅
exception_log	예외 상황 기록 및 관리자 알림	장애 분석, 관리자 대시보드

4. ENUM 타입 정의

```
1 CREATE TYPE order_status AS ENUM (  
2     'ORDER_RECEIVED',    -- 주문 접수  
3     'ORDER_WAIT',       -- 주문 대기  
4     'SORTING',           -- 상품 선별/상차 중  
5     'DELIVERING',        -- 상품 운반 중  
6     'INSPECTING',        -- 상품 검수/하차 중  
7     'PICKUP_READY',      -- 픽업 준비 완료  
8     'COMPLETED',        -- 수령 완료  
9     'ERROR'             -- 예외 발생  
10 );  
11  
12 CREATE TYPE order_item_status AS ENUM (  
13     'WAITING',           -- 작업 대기 중  
14     'SORTED',            -- 상품 선별 및 PICKY 적재 완료  
15     'INSPECTED',         -- 상품 검수 완료  
16     'MISSING',           -- 검수 결과 상품 누락  
17     'EXCESS',            -- 검수 결과 상품 초과  
18     'MISMATCH'          -- 검수 결과 불일치  
19 );  
20  
21 CREATE TYPE pickup_slot_status AS ENUM (  
22     'EMPTY',             -- 비어 있음  
23     'RESERVED',          -- 주문에 배정됨  
24     'OCCUPIED',          -- 상품 있음 / 고객 픽업 대기  
25     'BLOCKED'           -- 사용 불가  
26 );  
27  
28 CREATE TYPE robot_type AS ENUM (  
29     'PICKY',             -- 주행 로봇  
30     'COBOT'              -- 로봇팔  
31 );  
32  
33 CREATE TYPE robot_status AS ENUM (  
34     'OFFLINE',           -- 통신 단절  
35     'IDLE',              -- 대기 중 / 신규 작업 가능  
36     'BUSY',              -- 작업 수행 중 / 신규 작업 불가  
37     'CHARGING',          -- 충전 중  
38     'EMERGENCY_STOP',    -- 긴급 정지 상태  
39     'ERROR'              -- 로봇 오류 상태  
40 );  
41  
42 CREATE TYPE picky_state AS ENUM (  
43     'CHARGING',           -- 충전 중  
44     'STANDBY',            -- 대기/작업 가능 상태  
45     'MOVING_TO_PRODUCT',  -- 주문 상품 보관 구역으로 이동 중  
46     'WAITING_FOR_COBOT',  -- COBOT 작업 대기 중  
47     'MOVING_TO_PICKUP',   -- 픽업존으로 이동 중  
48     'MOVING_TO_STOCK',    -- 입고존으로 이동 중  
49     'MOVING_TO_STORAGE',  -- 상품 보관 구역으로 이동 중  
50     'RETURNING',          -- 대기/충전 구역 복귀 중  
51     'DOCKING',            -- 충전 도크 정밀 진입 중  
52     'ERROR_RECOVERY'     -- 주행 오류 복구 중  
53 );  
54  
55 CREATE TYPE cobot_state AS ENUM (  
56     'STANDBY',            -- 작업 시작 전 기본 상태  
57     'SORTING',            -- 카메라로 주문 상품을 찾고 선별 중  
58     'LOADING',            -- 선별한 상품을 PICKY에 상차 중  
59     'INSPECTING',         -- 상품 검수 중  
60     'UNLOADING',          -- 픽업 슬롯 하차 중  
61     'STOCKING_SORTING',   -- 입고존에서 대상 상품을 찾고 선별 중  
62     'STOCKING_LOADING',   -- 입고 상품을 PICKY에 상차 중  
63     'STOCKING_PLACING',   -- 입고 상품을 보관 위치에 적재 중  
64     'STOWING_ARM',        -- 로봇팔 기본 자세 복귀 중
```

```

65     'SAFETY_STOPPED'          -- 안전 정지
66 );
67
68 CREATE TYPE task_type AS ENUM (
69     'MOVE_TO_PRODUCT',        -- PICKY가 주문 상품 보관 구역으로 이동
70     'SORTING_AND_LOAD',       -- COBOT이 주문 상품을 집어 PICKY에 적재
71     'MOVE_TO_PICKUP',         -- PICKY가 픽업존으로 이동
72     'INSPECTION',             -- COBOT이 주문 상품 검수
73     'UNLOAD',                 -- COBOT이 픽업 슬롯에 상품 하차
74     'MOVE_TO_STOCK',          -- PICKY가 입고존으로 이동
75     'STOCKING_PICK',          -- COBOT이 입고존 상품 중 대상 상품을 집어
76     PICKY에 적재
77     'MOVE_TO_STORAGE',        -- PICKY가 해당 상품 보관 구역으로 이동
78     'STOCKING_PLACE',         -- COBOT이 대상 상품을 매대/보관 구역에 적재
79     'RETURN_HOME',           -- PICKY 대기/충전 구역 복귀
80     'CHARGE'                  -- PICKY 충전
81 );
82
83 CREATE TYPE task_status AS ENUM (
84     'QUEUED',                 -- 작업 큐 대기 중
85     'ASSIGNED',              -- 특정 로봇에 할당됨
86     'RUNNING',               -- 현재 수행 중
87     'PAUSED',                -- 긴급 정지 등으로 일시 중단
88     'SUCCESS',               -- 작업 성공 완료
89     'FAILED',                -- 작업 실패
90     'CANCELLED'              -- 작업 취소
91 );
92
93 CREATE TYPE exception_type AS ENUM (
94     'OBSTACLE_DETECTED',     -- 장애물 감지
95     'LOW_BATTERY',           -- 배터리 부족
96     'NAVIGATION_FAILED',     -- 경로 탐색 또는 주행 실패
97     'HARDWARE_ERROR',        -- 하드웨어 오류
98     'TIMEOUT',               -- 작업 시간 초과
99     'SORTING_FAIL',          -- 상품 선별 실패
100    'INSPECTION_FAIL',        -- 검수 실패
101    'HUMAN_DETECTED',         -- 사람 감지
102    'SYSTEM_ERROR'            -- 시스템 오류
103 );

```

5. 테이블 정의

5-1. zone

zone은 PICKY가 이동하거나 COBOT이 작업하는 주요 위치를 저장한다.

상품 보관 위치, 입고존, 픽업존, 충전 도크, 교통 제어 구역 등을 모두 zone으로 관리한다.

```

1 CREATE TABLE zone (
2     zone_id      SERIAL PRIMARY KEY,
3     zone_name    VARCHAR(50) NOT NULL,
4     zone_type    VARCHAR(30) NOT NULL,
5     pos_x        FLOAT NOT NULL,
6     pos_y        FLOAT NOT NULL,
7     pos_z        FLOAT NOT NULL,
8     pos_theta    FLOAT
9 );
10
11 INSERT INTO zone (zone_name, zone_type, pos_x, pos_y, pos_z,
12 pos_theta) VALUES
13 ('STANDBY_ZONE_1', 'STANDBY', 0.0, 0.0, 0.0, 0.0),
14 ('STANDBY_ZONE_2', 'STANDBY', 0.0, 0.0, 0.0, 0.0),
15 ('STOCK_ZONE', 'STOCK', 0.0, 0.0, 0.0, 0.0),
16 ('PRODUCT_ZONE_1', 'PRODUCT', 0.0, 0.0, 0.0, 0.0),
17 ('PRODUCT_ZONE_2', 'PRODUCT', 0.0, 0.0, 0.0, 0.0),

```

```

17 ('PRODUCT_ZONE_3', 'PRODUCT', 0.0, 0.0, 0.0, 0.0),
18 ('PRODUCT_ZONE_4', 'PRODUCT', 0.0, 0.0, 0.0, 0.0),
19 ('PRODUCT_ZONE_5', 'PRODUCT', 0.0, 0.0, 0.0, 0.0),
20 ('PRODUCT_ZONE_6', 'PRODUCT', 0.0, 0.0, 0.0, 0.0),
21 ('PICKUP_ZONE', 'PICKUP', 0.0, 0.0, 0.0, 0.0),
22 ('CHARGING_DOCK_1', 'CHARGING_DOCK', 0.0, 0.0, 0.0, 0.0),
23 ('CHARGING_DOCK_2', 'CHARGING_DOCK', 0.0, 0.0, 0.0, 0.0),
24 ('TRAFFIC_ZONE_1', 'TRAFFIC', 0.0, 0.0, 0.0, 0.0);

```

5-2. product

product는 상품 정보와 재고를 저장한다.

각 상품은 하나의 보관 zone에 연결된다.

```

1 CREATE TABLE product (
2     product_id SERIAL PRIMARY KEY,
3     name VARCHAR(100) NOT NULL,
4     image_url TEXT,
5     stock_qty INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0),
6     storage_zone_id INT NOT NULL REFERENCES zone(zone_id)
7 );

```

5-3. pickup_slot

pickup_slot은 고객이 상품을 가져갈 픽업 칸을 관리한다.

주문과의 연결은 orders.pickup_slot_id로 관리한다.

```

1 CREATE TABLE pickup_slot (
2     slot_id SERIAL PRIMARY KEY,
3     slot_name VARCHAR(50),
4     status pickup_slot_status NOT NULL DEFAULT 'EMPTY'
5 );
6
7 INSERT INTO pickup_slot (slot_name, status) VALUES
8     ('Pickup_slot_1', 'EMPTY'),
9     ('Pickup_slot_2', 'EMPTY'),
10    ('Pickup_slot_3', 'EMPTY'),
11    ('Pickup_slot_4', 'EMPTY');

```

5-4. robot_unit

robot_unit은 함께 동작하는 PICKY와 COBOT 묶음이다.

한 주문 또는 입고 작업은 하나의 robot_unit에 배정된다.

Fleet Manager는 unit_id를 기준으로 같은 세트의 PICKY와 COBOT을 찾는다.

```

1 CREATE TABLE robot_unit (
2     unit_id SERIAL PRIMARY KEY,
3     unit_name VARCHAR(50) NOT NULL,
4     description TEXT
5 );
6
7 INSERT INTO robot_unit (unit_name, description) VALUES
8     ('PICKY_UNIT_1', 'PICKY1 and COBOT1 pair'),
9     ('PICKY_UNIT_2', 'PICKY2 and COBOT2 pair');

```

5-5. orders

orders는 주문 1건의 대표 정보를 저장한다.

priority는 낮을수록 우선순위가 높다.

일반 주문의 기본 priority는 2이다.

assigned_unit_id는 해당 주문을 담당하는 robot_unit을 의미한다.

```
1 CREATE TABLE orders (  
2     order_id          SERIAL PRIMARY KEY,  
3     order_no          VARCHAR(30) UNIQUE,  
4     status            order_status NOT NULL DEFAULT 'ORDER_RECEIVED',  
5     priority          INT NOT NULL DEFAULT 2,  
6     pickup_slot_id    INT REFERENCES pickup_slot(slot_id),  
7     assigned_unit_id  INT REFERENCES robot_unit(unit_id)  
8 );
```

5-6. order_item

order_item은 주문 안에 들어 있는 상품 목록을 저장한다.

예를 들어 고객이 콜라 2개, 과자 1개를 주문하면 orders는 1개 생성되고, order_item은 2개 생성된다.

Fleet Manager는 order_item을 기준으로 상품별 task를 만든다.

```
1 CREATE TABLE order_item (  
2     item_id          SERIAL PRIMARY KEY,  
3     order_id         INT NOT NULL REFERENCES orders(order_id) ON DELETE  
4     CASCADE,  
5     product_id       INT NOT NULL REFERENCES product(product_id),  
6     quantity         INT NOT NULL CHECK (quantity > 0),  
7     status           order_item_status NOT NULL DEFAULT 'WAITING'  
8 );
```

5-7. robot

robot은 PICKY와 COBOT 개별 로봇의 현재 상태를 저장한다.

robot_status는 관제용 큰 상태이고, picky_state와 cobot_state는 타입별 세부 상태이다.

세부 상태가 없는 경우 picky_state 또는 cobot_state는 NULL로 둔다.

```
1 CREATE TABLE robot (  
2     robot_id         SERIAL PRIMARY KEY,  
3     robot_name       VARCHAR(30) UNIQUE NOT NULL,  
4     unit_id          INT REFERENCES robot_unit(unit_id),  
5     robot_type       robot_type NOT NULL,  
6     robot_status     robot_status NOT NULL DEFAULT 'IDLE',  
7     picky_state      picky_state,  
8     cobot_state      cobot_state,  
9     current_task_id  INT,  
10    ros_namespace    VARCHAR(50),  
11    battery_level    INT CHECK (battery_level BETWEEN 0 AND 100),  
12    pos_x            FLOAT,  
13    pos_y            FLOAT,  
14    pos_theta        FLOAT,  
15    CHECK (  
16        (robot_type = 'PICKY' AND cobot_state IS NULL)  
17        OR  
18        (robot_type = 'COBOT' AND picky_state IS NULL)
```

```

19 )
20 );
21
22 INSERT INTO robot (
23     robot_name,
24     unit_id,
25     robot_type,
26     robot_status,
27     picky_state,
28     cobot_state,
29     ros_namespace,
30     battery_level
31 ) VALUES
32     ('PICKY1', 1, 'PICKY', 'IDLE', NULL, NULL, '/picky1', 100),
33     ('COBOT1', 1, 'COBOT', 'IDLE', NULL, NULL, '/cobot1', NULL),
34     ('PICKY2', 2, 'PICKY', 'IDLE', NULL, NULL, '/picky2', 100),
35     ('COBOT2', 2, 'COBOT', 'IDLE', NULL, NULL, '/cobot2', NULL);

```

5-8. task

task는 Fleet Manager가 생성하고 관리하는 작업 단위이다.

orders가 고객 관점의 주문이라면, task는 로봇 실행 관점의 명령이다.

상품별 이동/상차 작업은 order_item_id를 기준으로 연결한다.

sequence_no는 단순 생성 순번이 아니라 Fleet Manager가 상품 위치, PICKY 현재 위치, 경로 계산 결과를 기준으로 정한 실행 순서이다.

입고 task의 경우 order_id와 order_item_id는 NULL일 수 있으며, 입고 대상 상품과 수량 정책은 payload_json에 저장한다.

```

1 CREATE TABLE task (
2     task_id          SERIAL PRIMARY KEY,
3     order_id         INT REFERENCES orders(order_id) ON DELETE
4     CASCADE,
5     order_item_id    INT REFERENCES order_item(item_id) ON DELETE
6     CASCADE,
7     sequence_no      INT NOT NULL,
8     assigned_robot_id INT REFERENCES robot(robot_id),
9     task_type        task_type NOT NULL,
10    status            task_status NOT NULL DEFAULT 'QUEUED',
11    priority          INT NOT NULL DEFAULT 2,
12    source_zone_id    INT REFERENCES zone(zone_id),
13    target_zone_id    INT REFERENCES zone(zone_id),
14    payload_json       JSONB,
15    result_message    TEXT
16 );
17
18 ALTER TABLE robot
19     ADD CONSTRAINT fk_robot_current_task
20     FOREIGN KEY (current_task_id) REFERENCES task(task_id);

```

5-9. task_event

task_event는 작업 상태 변화와 주요 이벤트를 기록한다.

작업 할당, 작업 시작, 경로 허가, 이동 완료, COBOT 작업 완료, 실패 사유 등을 남긴다.

```

1 CREATE TABLE task_event (
2     event_id          SERIAL PRIMARY KEY,

```

```

3      task_id      INT NOT NULL REFERENCES task(task_id) ON DELETE
      CASCADE,
4      robot_id     INT REFERENCES robot(robot_id),
5      from_status  task_status,
6      to_status    task_status NOT NULL,
7      event_name   VARCHAR(50),
8      reason       TEXT,
9      created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
10 );

```

5-10. exception_log

exception_log는 로봇이나 시스템에서 발생한 문제를 기록한다.

예외 발생 시점은 요구사항에 포함되어 있으므로 created_at을 유지한다.

```

1 CREATE TABLE exception_log (
2     exception_id SERIAL PRIMARY KEY,
3     robot_id     INT REFERENCES robot(robot_id),
4     task_id      INT REFERENCES task(task_id),
5     order_id     INT REFERENCES orders(order_id),
6     exception_type exception_type NOT NULL,
7     detail       TEXT,
8     is_resolved  BOOLEAN NOT NULL DEFAULT FALSE,
9     created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
10 );

```

6. 데이터 관계 요약

관계	설명
product.storage_zone_id → zone.zone_id	상품 보관 위치
orders.pickup_slot_id → pickup_slot.slot_id	주문 픽업 슬롯
orders.assigned_unit_id → robot_unit.unit_id	주문 담당 로봇 세트
order_item.order_id → orders.order_id	주문에 포함된 상품 목록
order_item.product_id → product.product_id	주문 상품 정보
robot.unit_id → robot_unit.unit_id	PICKY와 COBOT 세트 연결
robot.current_task_id → task.task_id	로봇이 현재 수행 중인 작업
task.order_id → orders.order_id	task가 속한 주문

task.order_item_id → order_item.item_id	상품별 task 연결
task.assigned_robot_id → robot.robot_id	task 담당 로봇
task.source_zone_id → zone.zone_id	작업 출발/기준 구역
task.target_zone_id → zone.zone_id	작업 목표 구역
task_event.task_id → task.task_id	작업 이벤트 대상 task
task_event.robot_id → robot.robot_id	작업 이벤트 발생 로봇
exception_log.robot_id → robot.robot_id	예외 발생 로봇
exception_log.task_id → task.task_id	예외 관련 task
exception_log.order_id → orders.order_id	예외 관련 주문

7. payload_json 사용 메모

task.payload_json은 task_type별 추가 데이터를 저장한다.

정형 컬럼으로 빼기 애매하거나 task마다 형태가 다른 데이터만 payload_json에 둔다.

입고 작업에서는 다음 정보를 저장할 수 있다.

```

1 {
2   "action": "STOCKING",
3   "product_id": 1,
4   "product_name": "콜라",
5   "requested_quantity": 5,
6   "detected_quantity": null,
7   "stocking_policy": "REQUESTED_QUANTITY"
8 }
```

필드	의미
requested_quantity	LLM 명령에서 파싱한 요청 입고 수량

detected_quantity	Vision 또는 COBOT이 실제 감지/적재한 수량
stocking_policy	REQUESTED_QUANTITY 또는 ALL_DETECTED
stock_delta	최종 재고 증가 수량이 필요할 때 사용